Aftermath Perpetuals Oracle

Security Assessment

January 6th, 2026 — Prepared by OtterSec

Michał Bochnak embe221ed@osec.io

Sangsoo Kang sangsoo@osec.io

Thiago Tavares thitav@osec.io

Table of Contents

Executive Summary **2**

Overview 2

Key Findings 2

Scope **3**

Findings **4**

Vulnerabilities **5**

OS-AMP-ADV-00 | Erroneous SLTP Size Clipping Logic 6

OS-AMP-ADV-01 | Missing Assistant Authorization Check 7

OS-AMP-ADV-02 | Invalid Overflow Assertion 8

General Findings **9**

OS-AMP-SUG-00 | Failure to Account for Mid-Session Fills 10

OS-AMP-SUG-01 | Code Validation 11

OS-AMP-SUG-02 | Code Maturity 14

Appendices

Vulnerability Rating Scale **16**

Procedure **17**

© 2025 Otter Audits LLC. All Rights Reserved.

01 — Executive Summary

Overview

Aftermath engaged OtterSec to assess the `perpetuals-v2`, `move-oracle-aggregator`, and `ifixed` programs. This assessment was conducted between November 14th and December 12th, 2025. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 6 findings throughout this audit engagement.

In particular, we identified a vulnerability in the function responsible for placing a stop order SLTP, where the size-clipping logic wrongly converts a negative base into a large u64, preventing proper capping of SLTP order size (OS-AMP-ADV-00). Additionally, we highlighted the lack of an assistant authorization check when creating a new FeedInfoObject (OS-AMP-ADV-01). Furthermore, the overflow check in ifixed incorrectly treats signed results as overflow by comparing the raw product against the greatest bit (OS-AMP-ADV-02).

We also recommended incorporating session deltas (filled bid/ask amounts) to validate reduce-only behavior correctly (OS-AMP-SUG-00), and advised to include proper validation logic to improve functionality and efficiency (OS-AMP-SUG-01). We further made suggestions regarding modifications to the codebase and ensuring adherence to coding best practices (OS-AMP-SUG-02).

© 2025 Otter Audits LLC. All Rights Reserved.

02 — Scope

The source code was delivered to us in Git repositories at:

1. github.com/AftermathFinance/perpetuals-v2
2. github.com/AftermathFinance/move-oracle-aggregator
3. github.com/AftermathFinance/move-ifixed

This audit was performed against commits 8165187, fb33a4b, and 28fcf25.

Brief descriptions of the programs are as follows:

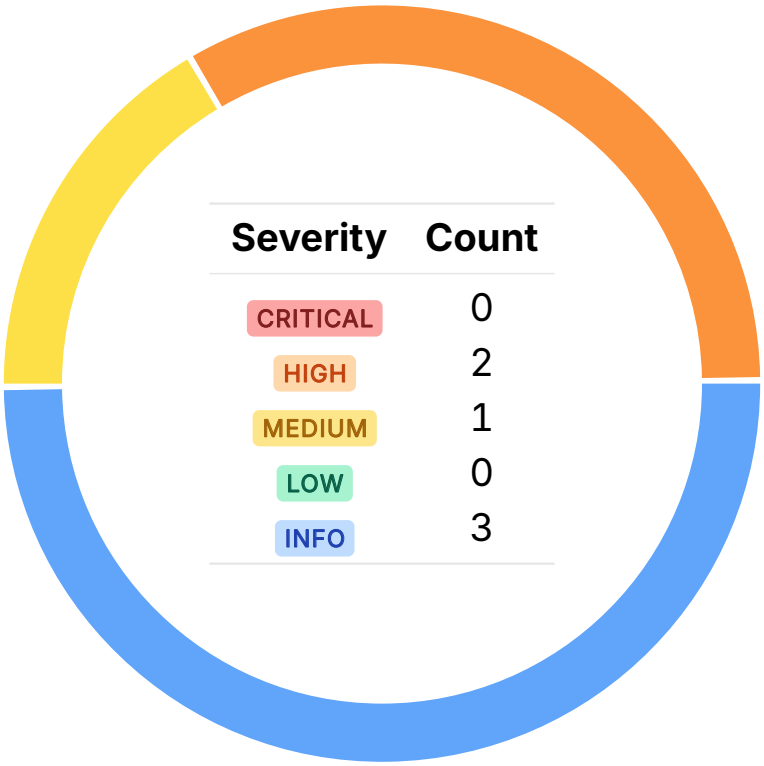
Name	Description
perpetuals-v2	An on-chain perpetual futures exchange that manages leveraged positions, order execution, margining, and liquidations. It integrates oracle aggregation, risk limits, and an insurance fund to maintain pricing integrity and system solvency.
move-oracle-aggregator	An on-chain price aggregation and validation layer used by the Perpetuals and Vault systems.
ifixed	It implements signed 18-decimal fixed-point arithmetic in Move on top of u256.

© 2025 Otter Audits LLC. All Rights Reserved.

03 — Findings

Overall, we reported 6 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	0
HIGH	2
MEDIUM	1
LOW	0
INFO	3

© 2025 Otter Audits LLC. All Rights Reserved.

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in Appendix A.

ID	Severity	Status	Description
OS-AMP-ADV-00	HIGH	RESOLVED	The size-clipping logic in `place_stop_order_sltp` wrongly converts a negative base into a huge `u64`, preventing proper capping of SLTP order size.
OS-AMP-ADV-01	HIGH	RESOLVED	`new` trusts any `AuthorityCap<PACKAGE, ASSISTANT>` without checking whether the holder is an authorized assistant.
OS-AMP-ADV-02	MEDIUM	RESOLVED	The overflow check in `ifixed` incorrectly treats signed results as overflow by comparing the raw product against `GREATEST_BIT`.

© 2025 Otter Audits LLC. All Rights Reserved.

Erroneous SLTP Size Clipping Logic HIGH OS-AMP-ADV-00

Description

The clipping logic in `clearing_house::place_stop_order_sltp` incorrectly calls `ifixed::to_balance(base, …)` even when `base` is negative. If `base` is negative when that call runs, `to_balance` will convert a negative fixed-point value into an unsigned `u64`. Thus, a signed fixed-point value is converted into a very large `u64`.

As the code utilizes `min(size, ifixed::to_balance(base, ...))` to clip the SLTP order to the position size, if `base < 0` and `to_balance(base, ...)` yields a huge `u64`, `min(size, huge_value)` will fail to cap the order size, allowing SLTP orders to exceed the actual position size.

>_ *perpetuals/sources/clearing_house.move* RUST

```rust
public(package) fun place_stop_order_sltp<T>([...]): (SessionSummary, Balance<SUI>,
    ↪  ClearingHouse<T>) {
    [...]
    else {
        place_market_order(
            &mut hot_potato,
            !position_is_ask,
            size,
            // We already calculated earlier the exact size to close,
            // so no need to pass `true` because the size is already clipped
            false
        );
    };
    [...]
}
```

Remediation

Explicitly assert that `base` is non-negative before converting it to a `balance`.

Patch

Resolved in 9754b1c.

© 2025 Otter Audits LLC. All Rights Reserved.

Missing Assistant Authorization Check HIGH OS-AMP-ADV-01

Description

`feed_info_object::new` does not verify whether the provided `AuthorityCap<PACKAGE, ASSISTANT>` corresponds to an authorized assistant listed in `config.active_assistants`. As a result, any holder of this capability may create arbitrary `FeedInfoObject` instances without approval. This breaks the intended access-control model and allows unauthorized feeds to be registered.

>_ sources/vault.move RUST

```rust
/// Create a `FeedInfoObject` for a specific Stork `feed_id`.
public fun new<Role>(
    wrapper: &mut OracleAggregatorStorkIntegration,
    cap: &AuthorityCap<PACKAGE, Role>,
    feed_id: vector<u8>,
): FeedInfoObject {
    let (wrapper_id, uid_derivation_key) = (
        wrapper.borrow_mut_id(cap),
        FeedInfoObjectKey(feed_id),
    );

    // i. Ensure that the there exists a 1:1 mapping between `feed_id: vector<u8>` to
    // `FeedInfoObject`.
    assert!(
        !derived_object::exists(wrapper_id, uid_derivation_key),
        EFeedInfoObjectAlreadyCreated,
    );

    FeedInfoObject {
        id: derived_object::claim(wrapper_id, uid_derivation_key),
        feed_id,
    }
}
```

Remediation

Ensure that the caller's `Role` is either `ADMIN` or `ASSISTANT`. If the caller is an assistant, verify that the assistant is listed in `config.active_assistants` before creating the object.

Patch

Resolved in eea5431.

© 2025 Otter Audits LLC. All Rights Reserved.

Invalid Overflow Assertion MEDIUM OS-AMP-ADV-02

Description

In `ifixed`, the `assert!(z <= GREATEST_BIT, OVERFLOW_ERROR)` check (in `from_u256balance`) and similar other checks assume that `z` exceeding `GREATEST_BIT` indicates overflow, but this is incorrect under the sign-magnitude encoding used by `ifixed`. When multiplying `balance * scaling_factor`, if one operand is negative, the raw multiplication naturally yields a value with the sign bit set, rendering `z > GREATEST_BIT` a valid intermediate state rather than an overflow. As a result, the assertion conflates sign representation with numeric overflow.

>_ *move-ifixed/packages/ifixed/sources/ifixed.move* RUST

```rust
/// Any-decimal balance to fixed-point number
public fun from_u256balance(balance: u256, scaling_factor: u256): u256 {
    let z = balance * scaling_factor;
    if (balance ^ scaling_factor < GREATEST_BIT) {
        assert!(z < GREATEST_BIT, OVERFLOW_ERROR);
        return z
    };
    assert!(z <= GREATEST_BIT, OVERFLOW_ERROR);
    (GREATEST_BIT - z) ^ GREATEST_BIT
}
```

Remediation

Remove unnecessary sign checks.

Patch

Resolved in 64f7dff.

© 2025 Otter Audits LLC. All Rights Reserved.

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-AMP-SUG-00	`check_reduce_only` only checks the starting position and ignores mid-session fills, resulting in valid reduce-only orders to be rejected.
OS-AMP-SUG-01	There are several instances where proper validation logic may be included to improve functionality and efficiency.
OS-AMP-SUG-02	Suggestions regarding inconsistencies in the codebase and ensuring adherence to coding best practices.

© 2025 Otter Audits LLC. All Rights Reserved.

Failure to Account for Mid-Session Fills OS-AMP-SUG-00

Description

`clearing_house::check_reduce_only` incorrectly relies solely on `position_base_before`, which reflects the position only at the start of the session. Because it ignores all fills that occur earlier in the same session, valid reduce-only orders may be rejected when a user first opens a position and then tries to reduce it immediately.

>_ *perpetuals/sources/clearing_house.move* RUST

```rust
fun check_reduce_only<T>(hot_potato: &SessionHotPotato<T>, side: bool, size: u64, reduce_only:
    ↪  bool): u64 {
    // If taker requires `reduce_only`, clip the size of the order to the position's size
    if (reduce_only) {
        let position_is_ask = ifixed::is_neg(hot_potato.position_base_before);
        // Check position's size is not 0 (nothing to reduce then) and the position's side
        // is the opposite of the order's side we are trying to place.
        assert!(
            hot_potato.position_base_before != 0 &&
            position_is_ask != side,
            errors::reduce_only_violated()
        );
        min(size, ifixed::to_balance(hot_potato.position_base_before, constants::b9_scaling()))
    } else {
        size
    }
}
```

Remediation

Incorporate session deltas (filled bid/ask amounts) to validate reduce-only behavior correctly.

Patch

Resolved in eea5431.

© 2025 Otter Audits LLC. All Rights Reserved.

Code Validation

OS-AMP-SUG-01

Description

1. `stop_orders::edit_stop_order_ticket_details` only checks that `stop_order_type` is within a numeric range, but does not validate whether the new type is semantically valid for the specific ticket or compatible with its existing configuration. As a result, incompatible stop-order types may be applied during edits. Validate `stop_order_type` in `edit_stop_order_ticket_details` .
2. In `authority::is_authorized` , utilize `exists_with_type` instead of `exists_` so the authorization check also enforces that the stored dynamic field matches the expected `AuthKey` type, ensuring more precise validation.

   >_ *packages/oracle-aggregator/sources/authority.move* RUST

   ```rust
   use fun is_authorized as UID.is_authorized;
   public fun is_authorized(id: &UID): bool {
       use fun sui::dynamic_field::exists_ as UID.exists;

       id.exists(AuthKey())
   }
   ```

3. In `clearing_house::compute_size_to_liquidate` , zero margin should be treated the same as negative margin. Utilize `less_than_eq` to ensure such cases trigger full liquidation.

   >_ *perpetuals/sources/clearing_house.move* RUST

   ```rust
   public(package) fun compute_size_to_liquidate([...]): (u256, bool) {
       [...]
       if (ifixed::less_than(margin_before_without_fc_fee, 0)) {
           // bad debt
           return (abs_b, false)
       };
       [...]
   }
   ```

4. The `price::valid_prices_from_sources_above_confidence` currently short-circuits only when `may_abort = true` , implying it will abort if no active sources exist. However, when `may_abort = false`, execution continues even if `source_ids` is empty, resulting in unnecessary processing and potentially misleading downstream behavior. Add an early return in the non-abort case for better consistency and to avoid unnecessary computation.

© 2025 Otter Audits LLC. All Rights Reserved.

>_ *perpetuals/sources/clearing_house.move* RUST

```rust
public fun valid_prices_from_sources_above_confidence(
    price_feed_storage: &PriceFeedStorage,
    config: &Config,
    source_ids: vector<ID>,
    confidence_bps: u64,
    staleness_threshold: u64,
    may_abort: bool,
    clock: &Clock,
): vector<u256> {
    // ia. Ensure that this package version can be interacted with.
    config.assert_package_version();

    // ib. Short-circuit if the `PriceFeedStorage` does not have any active sources.
    if (source_ids.length() == 0 && may_abort) abort ENoSources;
    [...]
}
```

5. Duplicate `source_ids` may result in the same oracle feed to be counted multiple times, distorting price calculations. Because the module currently assumes callers provide unique IDs, it is necessary to add an explicit uniqueness check for `source_ids: vector<ID>` to prevent double-counting and maintain correct pricing logic.
6. `price::newest_price_from_sources_above_confidence` compares timestamps before verifying confidence, allowing a newer but low-confidence price to overshadow an older, valid high-confidence price. Because the timestamp gate updates the selection logic even when confidence fails, the true newest valid price may never be chosen. Confidence should be checked before the timestamp comparison to ensure only eligible prices influence the selection.

>_ *packages/oracle-aggregator/sources/price.move* RUST

```rust
public fun newest_price_from_sources_above_confidence([...]): u256 {
    [...]
    source_ids.do!(|source_id| {
        let (price, confidence, _, timestamp) = price_feed_storage.as_parts!(source_id);

        // NOTE: If more than one price share the same timestamp, the first one is
            ↪  returned.
        //
        if (
            (timestamp.is_valid!(current_timestamp_ms, stale_timestamp_ms))
                && (max_timestamp_ms == 0xffff_ffff_ffff_ffff || timestamp >
                    ↪  max_timestamp_ms)
                && confidence >= confidence_threshold
        )
            (newest_price, max_timestamp_ms) = (price, timestamp);
    });
    [...]
}
```

© 2025 Otter Audits LLC. All Rights Reserved.

Remediation

Include the above validations.

Patch

1. Issue #1 resolved in 8228bed.
2. Issue #2 resolved in eea5431.
3. Issue #3 resolved in 8228bed.
4. Issue #4 resolved in eea5431.
5. Issue #5 resolved in eea5431.
6. Issue #6 resolved in eea5431.

© 2025 Otter Audits LLC. All Rights Reserved.

Code Maturity OS-AMP-SUG-02

Description

1. In `process_session_hot_potato`, the division to compute how much collateral must be seized or liquidated to cover a deficit (`missing_margin`) results in underestimating the required amount. Also, in `compute_size_to_liquidate`, the computation of `alpha` to determine the fraction of the position that must be liquidated to restore margin requirements utilizes floor-based `ifixed::div`, underestimating the value. Utilize ceil-rounded division to ensure the values are rounded up, favoring the protocol.
2. Currently, `ifixed::to_balance` assumes a non-negative input, but in several places it is invoked on values such as `base` that may be negative for short positions. Converting a negative fixed-point value into a `u64` balance is semantically invalid. Particularly in `place_stop_order_sltp`, when clipping the size of the order to the position's size. These cases should explicitly handle negative inputs before converting.

>_ *perpetuals/sources/clearing_house.move* RUST

```rust
public(package) fun place_stop_order_sltp<T>([...]): (SessionSummary, Balance<SUI>,
    ↪  ClearingHouse<T>) {
    [...]
    // Clip the size of the order to the position's size if greater
    let size = min(size, ifixed::to_balance(base, constants::b9_scaling()));
    [...]
}
```

3. The current comment for `price::price_is_valid` states that a price is valid if it is non-zero, but the implementation additionally requires it to be positive and below `ifixed::max_value`. Update the documentation to accurately describe these conditions to ensure consistency with the implementation.

>_ *packages/oracle-aggregator/sources/price.move* RUST

```rust
use fun price_is_valid as u256.is_valid;
// Returns true iff `price` is non-zero, false otherwise.
public macro fun price_is_valid(
    $price: u256,
): bool {
    // NOTE: Checks if price is positive --.
    //            .-----------------------^---.
    $price != 0 && $price < ifixed::max_value()
}
```

© 2025 Otter Audits LLC. All Rights Reserved.

4. `bcs::to_bytes` produces variable-length encodings for `Option<T>` in `verify_standalone_encrypted_details` within `clearing_house` since `none` and `some` serialize differently. While this currently does not result in mismatch because both hashing and verification utilize identical BCS encoding, it should nonetheless, remain consistent.

Remediation

Implement the above-mentioned suggestions.

Patch

1. Issue #1 resolved in 8228bed.
2. Issue #2 was acknowledged by the team.
3. Issue #3 resolved in eea5431.
4. Issue #4 resolved in 8228bed.

© 2025 Otter Audits LLC. All Rights Reserved.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2025 Otter Audits LLC. All Rights Reserved.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2025 Otter Audits LLC. All Rights Reserved.